

Module 6

Orchestration agentique (partie 1)

Patterns & LangGraph

Cursus AI Engineer · Neosoft · 1 jour

Positionnement du module

Vous avez désormais (M1-M5) :

- RAG complet et évalué (M5)
- Recherche hybride et re-ranking (M4)
- Préparation de corpus (M2)

Ce module ajoute la **dimension agentique** :

- L'agent **décide** dynamiquement quoi faire (vs. chaîne fixe)
- Boucles **Thought / Action / Observation**
- État partagé, branchements conditionnels, points de décision
- Premier agent **LangGraph** opérationnel

Avertissement : *pas tous les cas d'usage nécessitent un agent. On verra quand l'éviter.*

Objectifs pédagogiques

À l'issue du module, vous serez capable de :

1. **Distinguer** chaîne, workflow, et agent — savoir quand utiliser quoi
2. **Implémenter** un agent ReAct simple en LangGraph
3. **Concevoir** un graphe avec nodes, edges, conditional edges
4. **Définir** un State typé partagé
5. **Visualiser** le graphe d'exécution
6. **Identifier** les anti-patterns à éviter en production

*En **M7** : state management avancé, checkpointing, human-in-the-loop, CrewAI.*

Plan de la journée

Bloc	Contenu	Durée
Matin	Agent vs chaîne : quand choisir quoi ?	30 min
	Pattern ReAct détaillé	45 min
	Plan-and-Execute, Reflection	45 min
	Anti-patterns	30 min
Après-midi	LangGraph : philosophie & concepts	45 min
	Premier agent ReAct LangGraph	1h
	Conditional edges, visualisation	45 min
	TPs : agent multi-tools	1h

Partie 1

Qu'est-ce qu'un agent ?

Définition opérationnelle

Qu'est-ce qu'un agent LLM ?

Une boucle perception → raisonnement → action sur l'environnement



Boucle de feedback : tant que l'objectif n'est pas atteint

Différence avec une chaîne RAG : un agent **décide** dynamiquement quoi faire à chaque étape

Trois niveaux d'autonomie

Niveau	Description	Exemple
1. Chaîne	Séquence fixe d'étapes	RAG simple : retrieve → generate
2. Workflow conditionnel	Étapes fixes avec branchements	Si question = factuelle alors RAG, sinon LLM seul
3. Agent	LLM décide à chaque tour quoi faire	Boucle ReAct avec N outils

Plus le niveau d'autonomie est élevé, plus il faut de garde-fous, observabilité, tests.

Quand préférer une chaîne à un agent

✓ Préférer une chaîne quand :

- La séquence d'étapes est **prévisible** et stable
- La latence est **critique** (< 1s)
- Vous voulez tester de bout en bout (chaîne = déterministe)
- Le coût doit être **borné**

✗ Privilégier un agent quand :

- Le nombre d'étapes **dépend de la question**
- L'agent doit **explorer** un espace d'actions
- Plusieurs outils peuvent être nécessaires dans n'importe quel ordre
- Vous avez besoin de **réflexion / replanification**

*Règle d'or : **commencez par une chaîne**, passez à un agent si vraiment nécessaire.*

Coût et latence d'un agent

Un appel LLM simple : 1 prompt + 1 completion.

Un agent ReAct typique : 3 à 8 itérations avec à chaque tour :

- 1 LLM call (Thought + Action)
- 1 tool call (peut être lent : DB, API)
- 1 LLM call (intégration de l'observation)

Conséquences :

- Latence × 5-10 vs chaîne
- Coût × 3-8 vs chaîne
- Variance énorme : p50 peut être 2s, p95 30s

*Toujours mesurer la **distribution**, jamais juste la moyenne.*

Partie 2

Le pattern ReAct

ReAct expliqué

Pattern ReAct — Reasoning + Acting

Chaque tour : Thought → Action → Observation, jusqu'à Answer

Question : Combien d'utilisateurs actifs sur le tenant Acme et quel est leur plan ?

Itération 1

Thought

« Je dois d'abord identifier le tenant Acme, puis récupérer ses utilisateurs actifs. »

Action

```
find_tenant(name="Acme")
```

Observation

```
{tenant_id: "t_42", plan: "Pro"}
```



Itération 2

Thought

« Maintenant que j'ai t_42, je récupère les utilisateurs actifs sur ce tenant. »

Action

```
count_active_users(tenant_id="t_42")
```

Observation

```
{count: 87}
```

Itération 3 — Final answer

Thought

« J'ai tout ce qu'il me faut. Je peux répondre. »

Answer

Le tenant Acme a 87 utilisateurs actifs, plan Pro.

Forces : flexible, transparent, debugable. Limites : long contexte, peut diverger sans garde-fou.

Le prompt ReAct typique

Tu es un agent qui peut appeler des outils. Pour chaque tour :

1. ****Thought**** : raisonne sur ce qu'il faut faire ensuite
2. ****Action**** : appelle un outil avec des arguments JSON
3. ****Observation**** : tu reçois le résultat
4. Répète jusqu'à pouvoir donner Final Answer

Outils disponibles :

- find_tenant(name: str) → {id, plan}
- count_active_users(tenant_id: str) → {count}
- get_billing(tenant_id: str) → {amount, currency}

Question : {question}

*Aujourd'hui, le pattern est implémenté **nativement** par les SDK (function calling, tool use), plus besoin du parsing manuel.*

ReAct natif avec Anthropic tools

```
import anthropic

tools = [
    {
        "name": "find_tenant",
        "description": "Récupère un tenant par nom",
        "input_schema": {"type": "object", "properties": {"name": {"type": "string"}}, "required": ["name"]},
    },
    {"name": "count_active_users", "description": "...", "input_schema": {...}},
]

client = anthropic.Anthropic()
messages = [{"role": "user", "content": "Combien d'utilisateurs sur Acme ?"}]

while True:
    response = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=1024, tools=tools, messages=messages,
    )
    if response.stop_reason == "end_turn":
        break # Final answer atteint
    # Sinon : exécuter les tool_use blocks et ajouter les résultats
    ...
```

Forces du pattern ReAct

- ✓ **Transparent** : la trace Thought/Action est lisible humainement
- ✓ **Debugable** : on voit exactement quelle étape échoue
- ✓ **Flexible** : pas besoin de prévoir tous les flots à l'avance
- ✓ **Standard** : supporté par tous les SDK modernes (Anthropic, OpenAI, etc.)

Limites du pattern ReAct

- ✗ **Long contexte** : les itérations s'accumulent
- ✗ **Peut diverger** : explore inutilement, oublie l'objectif
- ✗ **Sensible au prompt** : 5% de variations perf selon formulation
- ✗ **Coûteux** : 3-8 LLM calls par requête

Partie 3

Plan-and-Execute & Reflection

Patterns avancés

Plan-and-Execute en pratique

Étape 1 : Planificateur (LLM fort, ex Sonnet)

```
PLAN_PROMPT = """Décompose cette tâche en 2-6 étapes ordonnées et indépendantes.  
Format JSON : [{"step": 1, "description": "...", "tool": "..."}]  
  
Tâche : {task}"""
```

Étape 2 : Exécuteur (LLM léger, ex Haiku)

```
for step in plan:  
    result = llm.execute(step.description, available_tools=[step.tool])  
    state[f"step_{step.step}_result"] = result
```

Étape 3 : Synthèse

```
final_answer = llm.synthesize(plan, all_step_results)
```

*Bénéfice : possibilité de **paralléliser** les étapes indépendantes.*

Reflection — auto-critique

```
GENERATE_PROMPT = "Réponds à : {question}\nContexte : {context}"
CRITIQUE_PROMPT = """Critique cette réponse :
- Faits non sourcés ?
- Information incomplète ?
- Hors-sujet ?
Liste les problèmes ou réponds OK si aucun."""
REVISE_PROMPT = "Voici la réponse initiale et les critiques. Régénère en corrigeant."

def reflect_loop(question, context, max_iter=3):
    answer = llm(GENERATE_PROMPT.format(question=question, context=context))
    for _ in range(max_iter):
        critique = llm(CRITIQUE_PROMPT + "\nRéponse : " + answer)
        if "OK" in critique:
            return answer
        answer = llm(REVISE_PROMPT + f"\n\nInitiale : {answer}\nCritiques : {critique}")
    return answer # max_iter atteint
```

Quand utiliser chaque pattern

Pattern	Cas d'usage
ReAct	Tâches exploratoires, nombre d'outils variable
Plan-and-Execute	Tâches longues, structurées, parallélisables
Reflection	Tâches sensibles à la qualité (rédaction, code)
Combiné	Plan global + ReAct dans chaque étape + Reflection finale

*En production : on assemble souvent ces patterns dans un **LangGraph** explicite.*

Partie 4

Anti-patterns

Les 4 anti-patterns à éviter

Anti-patterns d'agents — à éviter en production

✗ « Agent qui fait tout »

Un seul agent avec 30 tools, qui décide de tout en boucle libre.

Symptômes

- Latence imprévisible (3 à 60 sec)
- Coût qui explose sans alerting
- Comportement instable selon la T°
- Impossible à tester de bout en bout

→ Solution : workflows déterministes

✗ Pas de garde-fous

Boucle ReAct sans `max\_iterations`, sans budget tokens, sans timeout.

Risques

- Boucle infinie → facture salée
- Agent piégé en "réflexion" sans sortie
- Outils appelés en boucle sur la même donnée
- Pas d'escalade humaine quand bloqué

→ Solution : limites + monitoring + HIL

✗ Tools trop fins ou trop gros

Trop fin : `add(a, b)`, `multiply(a, b)`...

Trop gros : `manage\_user(action, payload)`

Symptômes

- Trop fin : 30 tool calls par tâche simple
- Trop gros : LLM se trompe d'arguments
- Description floue → mauvais routing

→ Solution : 1 tool = 1 action métier
avec description et schéma clairs

✗ Pas d'observabilité

Agent en prod sans tracing, sans logs structurés, sans dashboard.

Conséquences

- Bug en prod → impossible à reproduire
- Pas de mesure de qualité dans le temps
- Drift silencieux après changement modèle

→ Solution : Langfuse / Phoenix dès le J1
Approfondi en M9 (LLMOps)

Le bon dimensionnement d'un tool

Mauvais (trop fin) :

```
def add(a: int, b: int) -> int: ...  
def multiply(a: int, b: int) -> int: ...  
def divide(a: int, b: int) -> float: ...
```

Mauvais (trop gros) :

```
def manage_user(action: str, payload: dict) -> dict:  
    """action : create, update, delete, suspend, get, list..."""
```

Bon :

```
def get_user(user_id: str) -> User: ...  
def create_user(email: str, name: str, role: str) -> User: ...  
def suspend_user(user_id: str, reason: str) -> bool: ...
```

*Un tool = une **action métier** avec une responsabilité claire et des paramètres typés.*

Garde-fous indispensables

```
# 1. Limite d'itérations
MAX_ITERATIONS = 10

# 2. Budget tokens par session
MAX_TOKENS_PER_SESSION = 50_000

# 3. Timeout par tool
TOOL_TIMEOUT_SECONDS = 30

# 4. Détection de boucle (mêmes args appelés N fois)
def detect_loop(history, current_call, max_repeat=3):
    repeats = sum(1 for h in history if h.tool == current_call.tool and h.args == current_call.args)
    return repeats >= max_repeat

# 5. Escalade humaine si bloqué
if iterations >= MAX_ITERATIONS or detect_loop(history, current):
    raise EscalationRequired("Agent stuck, needs human")
```

Partie 5

LangGraph

Pourquoi LangGraph et pas LangChain Agents ?

LangChain Agent (legacy) :

- Boucle ReAct cachée derrière une abstraction
- Difficile de déboguer, customiser, tester
- Approche "magique"

LangGraph (recommandé) :

- Graphe **explicite** de nodes et edges
- État typé partagé
- Visualisation native du flot
- Checkpointing pour persistance / reprise
- Approche "code-first"

*En 2026, **LangGraph** est le standard de fait pour orchestrer des agents production.*

Concepts clés

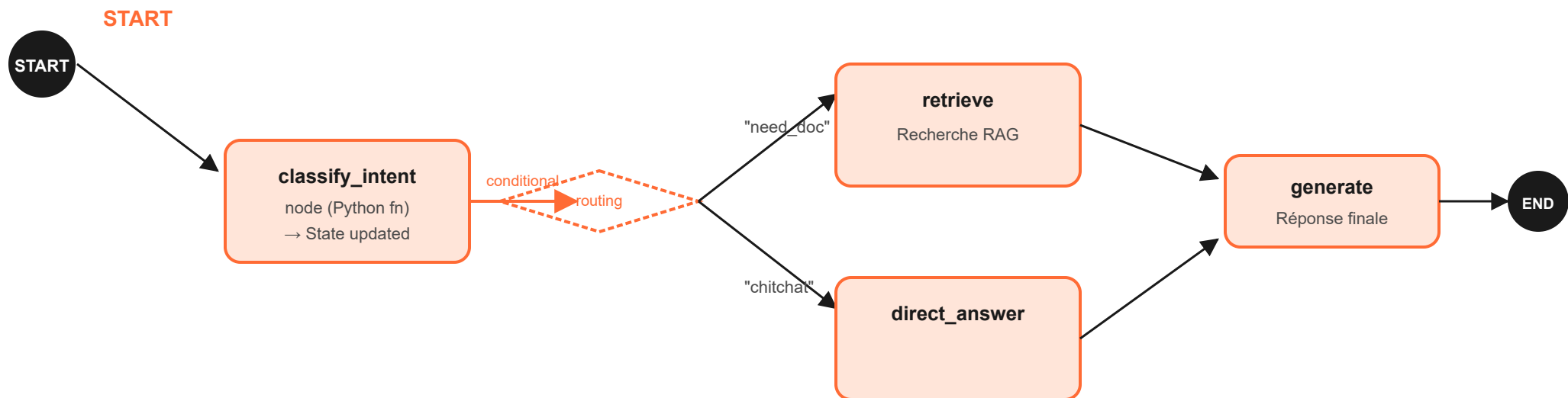
LangGraph — concepts clés

State

Dict typé partagé entre tous les nodes. Chaque node lit/modifie l'état.

```
class State(TypedDict): question: str; answer: str; messages: Annotated[list, add_messages]
```

Persistence via checkpointing (SQLite, PostgreSQL, Redis) → reprise possible.



Légende

 Node : fonction Python qui lit et modifie le state

 Edge : transition (toujours empruntée)

 Conditional edge : routing dynamique

State : la mémoire partagée

```
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    question: str
    intent: str | None
    retrieved: list[dict]
    answer: str | None
    messages: Annotated[list, add_messages] # accumule au lieu de remplacer
    iteration: int
```

Règle d'écriture :

- Par défaut, un node retourne un dict qui **remplace** la valeur dans l'état
- Avec `Annotated[..., reducer_fn]`, on **accumule** (utile pour les messages)

Premier agent LangGraph — squelette

```
from langgraph.graph import StateGraph, START, END

def classify_intent(state: AgentState) -> dict:
    intent = llm.classify(state["question"])
    return {"intent": intent}

def retrieve(state: AgentState) -> dict:
    docs = retriever.search(state["question"])
    return {"retrieved": docs}

def generate(state: AgentState) -> dict:
    answer = llm.generate(state["question"], state["retrieved"])
    return {"answer": answer}

graph = StateGraph(AgentState)
graph.add_node("classify", classify_intent)
graph.add_node("retrieve", retrieve)
graph.add_node("generate", generate)

graph.add_edge(START, "classify")
graph.add_edge("classify", "retrieve")
graph.add_edge("retrieve", "generate")
graph.add_edge("generate", END)

agent = graph.compile()
```

Conditional edges — branchements dynamiques

```
def route_intent(state: AgentState) -> str:
    """Retourne le nom du prochain node."""
    if state["intent"] == "factual":
        return "retrieve"
    elif state["intent"] == "chitchat":
        return "direct_answer"
    else:
        return "escalate_human"

graph.add_conditional_edges(
    "classify",
    route_intent,
    {
        "retrieve": "retrieve",
        "direct_answer": "direct_answer",
        "escalate_human": "escalate_human",
    },
)
```

Les `conditional_edges` permettent au graphe de prendre des décisions sans LLM "décideur".

Visualiser le graphe

```
from IPython.display import Image
img = agent.get_graph().draw_mermaid_png()
# ou en CLI
print(agent.get_graph().draw_ascii())
```

```
graph TD
    START --> classify
    classify -- factual --> retrieve
    classify -- chitchat --> direct_answer
    retrieve --> generate
    direct_answer --> generate
    generate --> END
```

Toujours **commiter** une visualisation du graphe avec le code → revue de PR plus claire.

Implémentation tool-using agent (complet)

```
from langgraph.prebuilt import create_react_agent
from langchain_anthropic import ChatAnthropic

@tool
def get_user(user_id: str) -> dict:
    """Récupère un utilisateur par ID."""
    return db.users.get(user_id)

@tool
def list_tickets(user_id: str, status: str = "open") -> list[dict]:
    """Liste les tickets d'un utilisateur."""
    return db.tickets.filter(user_id=user_id, status=status)

llm = ChatAnthropic(model="claude-sonnet-4-6", temperature=0)
agent = create_react_agent(llm, tools=[get_user, list_tickets])

result = agent.invoke({
    "messages": [("user", "Combien de tickets ouverts pour user_42 ?")]
})
```

`create_react_agent` est un raccourci LangGraph pour le pattern ReAct standard.

Streaming des étapes intermédiaires

```
for chunk in agent.stream(
    {"messages": [("user", "Question complexe...)]}],
    stream_mode="values", # ou "updates"
):
    if "messages" in chunk:
        last = chunk["messages"][-1]
        if last.type == "tool":
            print(f"🔧 Tool result: {last.content[:80]}...")
        elif last.type == "ai" and last.tool_calls:
            for call in last.tool_calls:
                print(f"😬 Calling {call['name']}({call['args']})")
        elif last.type == "ai":
            print(f"🗣️ {last.content}")
```

Le streaming est essentiel pour l'UX : l'utilisateur voit l'agent travailler.

Tests d'agents — comment faire

Niveau 1 : tests unitaires des nodes

```
def test_classify_intent():
    state = {"question": "Bonjour ça va ?"}
    out = classify_intent(state)
    assert out["intent"] == "chitchat"
```

Niveau 2 : tests du graphe sans LLM (mocks)

```
def test_graph_routing(mock_llm):
    mock_llm.classify.return_value = "factual"
    result = agent.invoke({"question": "Combien d'users ?"})
    assert "retrieve" in result["nodes_visited"]
```

Niveau 3 : tests end-to-end avec golden set + Ragas

- 20-30 cas réels avec réponse attendue
- Faithfulness > 0.85, time-to-answer < seuil

Partie 6

TPs

TP 1 — Premier agent LangGraph (45 min)

Objectif : implémenter un agent qui répond à des questions support en utilisant 3 tools (RAG, lookup utilisateur, créer ticket).

→ Voir `exercices/exo1_first_agent.md`

TP 2 — Routing dynamique avec conditional edges (30 min)

Objectif : ajouter un nœud de classification d'intent + 3 chemins distincts.

→ Voir `exercises/exo2_routing.md`

Récapitulatif de la journée

- ✓ Distinguer chaîne, workflow, agent
- ✓ Comprendre les patterns ReAct, Plan-and-Execute, Reflection
- ✓ Identifier les anti-patterns d'agents en production
- ✓ Construire un agent LangGraph avec State, nodes, edges
- ✓ Implémenter des conditional edges pour router dynamiquement
- ✓ Visualiser et tester un agent

Prochain module (M7) : state management avancé, checkpointing, human-in-the-loop, comparaison avec CrewAI.

Questions ?

Merci pour votre attention

Support, démos et exercices : `supports/M06_agents_partie1/`